

ZKP Use-Case Study

Succinct's Platform, Prover Network and SP1

Philippe Laporte

Recent Developments 6630
Information Systems Engineering
Concordia University

March 22 2026

1. What is your student number? What is the episode number?

24172957/314

2. Who is the guest and what is their background in ZKP?

Uma Roy, CEO and cofounder of [Succinct](#). An MIT graduate, she has deep expertise in applied ZKPs.

3. Briefly introduce what the use-case is about

There are 3 uses cases. SP1 is a zkVM that transforms any RUST program into a ZKP proof. The Succinct platform provides integrated proving services, and last, the Prover Network provides a marketplace for such services.

Let us focus on SP1 for the rest of the Case Study.

SP1 is a 100% open-source RISC-V zkVM (Zero Knowledge Virtual Machine). In simplest terms: you write normal Rust code, and SP1 produces a ZK proof of its correct execution.

One major use case of zkVM-produced proofs is for on-chain (e.g. on the Ethereum Blockchain) verification, and thus has to be low on resource requirements .

4. What is the timestamp of where the use-case is first mentioned?

SP1 is mentioned at 04:18

5. Provide the necessary background to understand what problem the use-case is trying to solve

After building the Succinct Platform, the team realized the biggest remaining bottleneck was that nobody wants to write ZK circuits. Even with great tooling around deployment and proof management, if the circuit itself takes months of specialized work to produce, adoption stays extremely limited. Uma estimates fewer than 1,000 people in the world have ever written a ZK circuit, versus millions of Rust developers. SP1 is the answer to that gap and remove the need to write circuits entirely by letting developers write normal Rust instead.

The team also wanted to use a ZKVM internally for their own bridge work, but found that the only real option at the time - RISC Zero - was a non-starter because its core constraint logic was closed-source and therefore not modifiable. For performance-sensitive use cases, they needed to be able to customize and optimize the VM. Since nothing suitable existed, they built it themselves.

6. A ZKP proof proves one (or more) specific statement, that in class, we think of as a function (or circuit). For the use-case, answer the following questions.

SP1 is a general-purpose zkVM, meaning the "function" it proves is not one specific computation but rather the correct execution of any arbitrary Rust program. So the answers below describe the underlying circuit that SP1 uses to prove that a RISC-V program ran correctly.

6a) What are the inputs to the function?

Two categories:

- Public inputs:
 - The program itself: the RISC-V bytecode being executed
 - The public inputs to the program (whatever the program is supposed to receive as public data)
 - The claimed output of the program
- Private inputs (witnesses):
 - The private inputs to the program (whatever secret data the program operates on, if any)
 - The full execution trace - meaning every intermediate state of the RISC-V machine as it runs: every register value, every memory read/write, every instruction executed, step by step

The execution trace is the key private witness. It's what allows the prover to convince the verifier that the program ran correctly without the verifier having to re-execute the whole thing.

6b) Is this a zk-SNARK or just a SNARK?

This depends on how SP1 is being used:

- In the general case, it is a zk-SNARK - the system is designed to support private inputs, and the execution trace (which may contain sensitive data) is kept hidden from the verifier.
- However, for many of Succinct's own use-cases, there are no private inputs - all the data being proven (block headers, validator signatures) is public. In those cases it functions as a plain SNARK, using only the succinctness and verifiability properties, not the zero-knowledge property.

6b) What are the outputs of the function?

- The proof itself - a succinct cryptographic proof that the program was executed correctly
- The public outputs of the program - whatever the Rust program was designed to output (e.g., a verified block hash in Blockchain)

The verifier receives both the proof and the public outputs, and can check that the program with the committed code and public inputs genuinely produced those public outputs, without re-executing the program.

6c) What does the function do?

The SP1 circuit takes the RISC-V bytecode of a Rust program together with its inputs, and verifies that the program's execution trace is valid. More precisely, it checks:

1. The first state of the execution trace matches the program's starting conditions (correct program loaded, correct inputs, registers and memory initialized properly)
2. Every subsequent state in the trace follows correctly from the previous one according to the RISC-V instruction set rules - i.e., each instruction was decoded and applied correctly, registers updated correctly, memory read and written correctly

3. Any pre-compile operations (e.g., hash function calls, elliptic curve operations) were computed correctly, using optimized sub-circuits for those specific operations
4. The final state of the trace matches the claimed output

If all of these checks pass, the proof certifies: "this specific Rust program, given these inputs, produced these outputs, via a valid execution." The verifier never needs to see the execution trace or re-run the program - they just check the proof.

6d) Is it important that the proof size is small (it is always nice, but is it necessary?)

Yes, it is necessary - and this is central to SP1's entire purpose. The whole point of using a zkVM instead of just running the program is that the proof can be verified cheaply on-chain. If proof sizes were large, on-chain verification would cost prohibitive amounts of gas, making SP1 useless for its primary use cases (ZK bridges, ZK rollups, etc.). Succinctness is not a nice-to-have, it is the core property that makes the system useful. In fact it's right there in the company name 😊

Additionally, since proofs may be aggregated (many proofs combined into one), small individual proof sizes are important to make aggregation tractable.

6e) Is it important that the verifier can check the proof without much computational effort (it is always nice, but is it necessary?)

Yes, it is necessary for the same reasons as above. The verifier is typically an Ethereum smart contract operating under severe gas constraints. The entire value proposition of SP1 is that heavy computation happens off-chain at the prover side, and only a lightweight check happens on-chain.

6f) How reasonable is it that the prover would want to spend extra time creating a proof for this use-case?

Very reasonable, for these reasons:

- The alternative is worse: Without SP1, the developer would have to spend months writing a custom ZK circuit for their specific computation. Spending extra proving time (minutes to hours on capable hardware) is trivially acceptable compared to months of circuit engineering.
- Proving is off-chain: The prover does their work off-chain, so latency is much more tolerable than on-chain computation. Most applications can absorb proving times of minutes without issue.

- Economic incentive: In the context of the prover network, provers are compensated for their work via the auction mechanism. The extra time spent proving translates directly into revenue, so there is a clear economic motivation.
 - Amortization: For recurring proofs the proving infrastructure is set up once and runs continuously, so the overhead is just an operational cost that gets amortized across many proofs.
 - SUCCINT's own experience: The interviewee says that internally, tasks that took months now take an afternoon - the proving time overhead is clearly not a concern compared to the development time savings.
7. Podcast guests are often going to sell or hype their technology or research. Think critically about any limitations to the suggested use-case. It might be hard to get the right people to deploy, hard to use, heavier on trust, problems with the law or regulation, expensive, etc. Discuss the limitations (or justify why there are no obvious ones).

First, performance is still a genuine concern - the VM abstraction adds overhead over hand-written circuits. GPU support isn't there yet, which matters because GPU acceleration is increasingly where proof generation performance is won.

Second, the "just write Rust" pitch somewhat undersells the remaining complexity - developers still need to understand what makes a computation ZK-friendly (e.g., avoiding non-determinism, understanding public vs. private inputs, knowing what goes on-chain), which is non-trivial even if circuit writing is eliminated.

Third, the security audit situation is concerning - SP1 was just released at time of recording, the core circuit logic had not yet been audited, and a bug in the VM's constraint system would be catastrophic and universal, affecting every single application built on top of it simultaneously, unlike a bug in a bespoke circuit which only affects one application.

Fourth, the open-source positioning, while genuinely valuable, also means anyone can fork it and fragment the ecosystem, potentially splitting the developer community and the prover network demand that SP1 is supposed to generate.

Finally, many applications have latency requirements that current proving times simply cannot meet (such as general-usage GPTs), and the economics of proof generation may make sense for low-value or high-frequency use cases.

8. Propose one realistic improvement or alternative design choice (e.g., change the statement, reduce trust, simplify with signatures/attestations, different ZK primitive, better UX, etc)

One realistic and high-impact improvement would be to invest heavily in a Rust-native developer feedback loop - specifically, a fast local simulation mode that executes the Rust program natively (without generating a proof) and produces detailed profiling information showing which parts of the program are expensive in terms of proof generation, which pre-compiles are being triggered, and where the computational bottlenecks are. Right now, a developer writes Rust, sends it to the prover, waits potentially minutes for a proof, and only then discovers their code is slow or uses unsupported operations. This is a painful iteration cycle. A lightweight local tool that gives instant feedback - essentially a "ZK linter" or "ZK profiler" - would dramatically improve the developer experience and make the "just write Rust" promise feel truly seamless. This doesn't require any changes to the underlying cryptography or circuit design, making it a realistic near-term engineering investment, and it would directly address the gap between SP1's pitch of accessibility and the reality that ZK-aware programming still has hidden gotchas that only surface at proving time.

9. State whether you used AI or not to help with your assignment, and if you did, include a link to the transcript.

I did use it to research [across different agents] which podcast to choose given certain aspects of my class project:

<https://claude.ai/share/fcbd790b-e39b-4380-a800-c9d3cbcb3543>

<https://chatgpt.com/share/69c059f7-491c-8013-9dd7-3e9cfcaeb8bc>

Also one question in another conversation: which podcast is most relevant to the project we are working on? [yet undisclosed results as those would be too revealing at this stage 😊]

Then I used it to research referenced concepts:

<https://claude.ai/share/63d1a15e-0392-44d6-a0c3-3094d2fb821b>